



C++ TUTORIAL

For beginners

This comprehensive C++ tutorial is designed for beginners who are eager to embark on their programming journey. Throughout the course, we will explore the fundamental concepts of C++, including data types, control structures, functions, and object-oriented programming principles. Each section is structured to build upon the previous one, ensuring a smooth learning curve. With practical examples and hands-on exercises, you will gain confidence in writing and debugging C++ code. By the end of this tutorial, participants will have a solid foundation in C++ programming, enabling them to tackle more advanced topics and projects

Smart Freshman Tutorials

C++ Tutorial

Learn C++

C++ is a popular programming language.

C++ is used to create computer programs, and is one of the most used language in game development.

C++ was developed as an extension of [C](#), and both languages have almost the same syntax.

Example

```
#include <iostream>
using namespace std;
```

```
int main() { cout << "Hello
World!";
return 0;
}
```

Introduction

What is C++?

C++ is a cross-platform language that can be used to create high-performance applications.

C++ was developed by [Bjarne Stroustrup](#), as an extension to the C language.

C++ gives programmers a high level of control over system resources and memory.

The language was updated 5 major times in [2011](#), [2014](#), [2017](#), [2020](#), and [2023](#) to C++11, C++14, C++17, C++20, and C++23.

Why Use C++?

C++ is one of the world's most popular programming languages.

C++ can be found in today's operating systems, Graphical User Interfaces, and embedded systems.

C++ is an object-oriented programming language which gives a clear structure to programs and allows code to be reused, lowering development costs.

C++ is portable and can be used to develop applications that can be adapted to multiple platforms.

C++ is fun and easy to learn!

As C++ is close to [C](#), [C#](#) and [Java](#), it makes it easy for programmers to switch to [C++](#) or vice versa.

Difference between C and C++

C++ was developed as an extension of C, and both languages have almost the same syntax.

The main difference between C and C++ is that C++ supports classes and objects, while C does not.

Get Started

When you are finished with this tutorial, you will be able to write C++ programs and create real-life examples.

It is not necessary to have any prior programming experience.

Sample Code:

```
#include <iostream>
using namespace std;

int main() { cout << "Hello
World!";
return 0;
}
```

Result:

```
Hello World!
```

This editor will be used in the entire tutorial to demonstrate the different aspects of C++.

Install C++

If you want to run C++ on your own computer, you need two things:

- A text editor, like Notepad, to write C++ code
- A compiler, like GCC, to translate the C++ code into a language that the computer will understand

There are many text editors and compilers to choose from. In the next steps, we will show you how to use an **IDE** that includes both.

Install C++ IDE

An IDE (Integrated Development Environment) is used to edit AND compile the code.

Popular IDE's include Code::Blocks, Eclipse, and Visual Studio. These are all free, and they can be used to both edit and debug C++ code.

Note: Web-based IDE's can work as well, but functionality is limited.

We will use **Code::Blocks** in our tutorial, which we believe is a good place to start.

You can find the latest version of Codeblocks at <http://www.codeblocks.org/>. Download the **mingw-setup.exe** file, which will install the text editor with a compiler.

C++ Quick start

Let's create our first C++ file.

Open Codeblocks and go to **File > New > Empty File**.

Write the following C++ code and save the file as **myfirstprogram.cpp** (**File > Save File as**):

```
myfirstprogram.cpp
```

```
#include <iostream>  
using namespace std;
```

```
int main() { cout << "Hello
World!";
return 0;
}
```

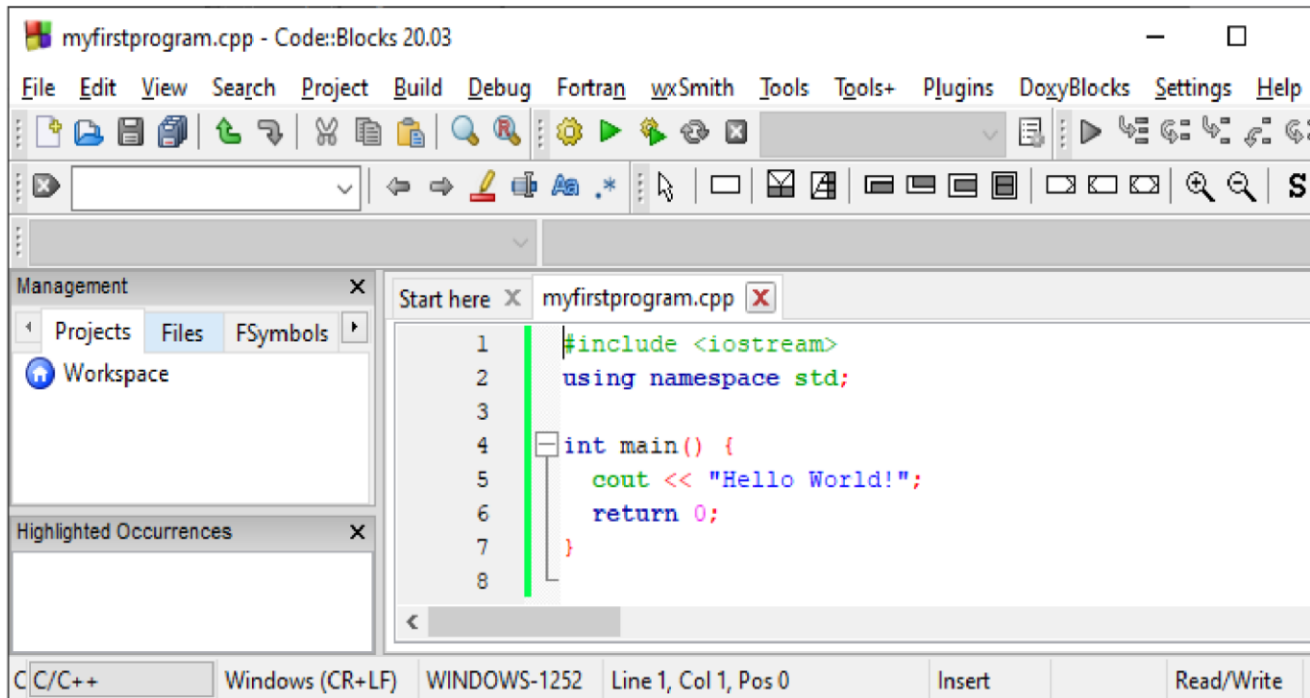
Don't worry if you don't understand the code above - we will discuss it in detail in later chapters. For now, focus on how to run the code.

In Codeblocks, it should look like this:

Then, go to **Build > Build and Run** to run (execute) the program. The result will look something to this:

```
Hello World!
Process returned 0 (0x0) execution time : 0.011 s
Press any key to continue.
```

Congratulations ! You have now written and executed your first C++ program .



C++ Syntax

Let's break up the following code to understand it better:

Example

```
#include <iostream>
using namespace std;
```

```
int main() { cout << "Hello
World!";
return 0;
}
```

Example explained

Line 1: `#include <iostream>` is a **header file library** that lets us work with input and output objects, such as `cout` (used in line 5). Header files add functionality to C++ programs.

Line 2: `using namespace std` means that we can use names for objects and variables from the standard library.

Don't worry if you don't understand how `#include <iostream>` and `using namespace std` works. Just think of it as something that (almost) always appears in your program.

Line 3: A blank line. C++ ignores white space. But we use it to make the code more readable.

Line 4: Another thing that always appear in a C++ program is `int main()`. This is called a **function**. Any code inside its curly brackets `{ }` will be executed.

Line 5: `cout` (pronounced "see-out") is an **object** used together with the *insertion operator* (`<<`) to output/print text. In our example, it will output "Hello World!".

Note: C++ is case-sensitive: "cout" and "Cout" has different meaning.

Note: Every C++ statement ends with a semicolon `;`.

Note: The body of `int main()` could also been written as: `int main() { cout << "Hello World! "; return 0; }` **Remember:** The compiler ignores white spaces. However, multiple lines makes the code more readable.

Line 6: `return 0;` ends the main function.

Line 7: Do not forget to add the closing curly bracket `}` to actually end the main function.

Omitting Namespace

You might see some C++ programs that runs without the standard namespace library. The `using namespace std` line can be omitted and replaced with the `std` keyword, followed by the `::` operator for some objects:

```
#include <iostream>
```

Example

```
int main() {  
    std::cout << "Hello World!";  
    return 0;  
}
```

It is up to you if you want to include the standard namespace library or not.

C++ Statements

A **computer program** is a list of "instructions" to be "executed" by a computer.

In a programming language, these programming instructions are called **statements**.

The following statement "instructs" the compiler to print the text "Hello World" to the screen:

Example

```
cout << "Hello World!";
```

It is important that you end the statement with a semicolon ;

If you forget the semicolon (;), an error will occur and the program will not run:

Example

```
cout << "Hello World!"
```

error: expected ';' before 'return'

Many Statements

Most C++ programs contain many statements.

The statements are executed, one by one, in the same order as they are written:

Example

```
cout << "Hello World!"; cout << "Have a good day!"; return 0;
```

Example explained

From the example above, we have three statements:

1. `cout << "Hello World!";`
2. `cout << "Have a good day!";`
3. `return 0;`

The first statement is executed first (print "Hello World!" to the screen).

Then the second statement is executed (print "Have a good day!" to the screen). And at last, the third statement is executed (end the C++ program successfully).

You will learn more about statements while reading this tutorial. For now, just remember to always end them with a semicolon to avoid any errors.

Coming up: The next chapter will teach you how to control the output and how to insert new lines to make it more readable.

C++ Output (Print Text)

The `cout` object, together with the `<<` operator, is used to output values and print text.

Just remember to surround the text with double quotes (""):

Example

```
#include <iostream>
using namespace std;
```

```
int main() {
```

https://t.me/Freshman_tutorials


```
cout << "Hello World!";  
return 0;  
}
```

You can add as many `cout` objects as you want. However, note that it does not insert a new line at the end of the output:

Example

```
#include <iostream>  
using namespace std;  
  
int main() { cout << "Hello  
World!"; cout << "I am  
learning C++"; return 0;  
}
```

C++ Output (Print) Numbers

You can also use `cout()` to print numbers.

However, unlike text, we don't put numbers inside double quotes:

Example

```
#include <iostream>  
using namespace std;  
  
int main() {  
  
    cout << 3; return  
0;  
}
```

You can also perform mathematical calculations:

Example

```
cout << 3 + 3;
```

Example

```
cout << 2 * 5;
```

Like and share!!